Python - Add Set Items

Add Set Items

Adding set items implies including new elements into an existing set. In Python, sets are mutable, which means you can modify them after they have been created. While the elements within a set must be immutable (such as integers, strings, or tuples), the set itself can be modified.

You can add items to a set using various methods, such as add(), update(), or set operations like union (|) and set comprehension.

> *One of the defining features of sets is their ability to hold only immutable (hashable) objects. This is because sets internally use a hash table for fast membership testing. Immutable objects are hashable, meaning they have a hash value that never changes during their lifetime.*

Add Set Items Using the add() Method

The add() method in Python is used to add a single element to the set. It modifies the set by inserting the specified element if it is not already present. If the element is already in the set, the add() method has no change in the set.

Syntax

Following is the syntax to add an element to a set −

```
set.add(obj)
```

Where, **obj** is an object of any immutable type.

Example

In the following example, we are initializing an empty set called "language" and adding elements to it using the add() method −

```
# Defining an empty set
language = set()

# Adding elements to the set using add() method
language.add("C")
language.add("C++")
language.add("Java")
language.add("Python")

# Printing the updated set
print("Updated Set:", language)
```

It will produce the following output −

```
Updated Set: {'Python', 'Java', 'C++', 'C'}
```

Add Set Items Using the update() Method

In Python, the update() method of set class is used to add multiple elements to the set. It modifies the set by adding elements from an iterable (such as another set, list, tuple, or string) to the current set. The elements in the iterable are inserted into the set if they are not already present.

Syntax

Following is the syntax to update an element to a set −

```
set.update(obj)
```

Where, **obj** is a set or a sequence object (list, tuple, string).

Example: Adding a Single Set Item

In the example below, we initialize a set called "my_set". Then, we use the update() method to add the element "4" to the set −

```
# Define a set
my_set = {1, 2, 3}

# Adding element to the set
my_set.update([4])
# Print the updated set
print("Updated Set:", my_set)
```

Following is the output of the above code −

```
Updated Set: {1, 2, 3, 4}
```

Example: Adding any Sequence Object as Set Items

The update() method also accepts any sequence object as argument.

In this example, we first define a set and a tuple, lang1 and lang2, containing different programming languages. Then, we add all elements from "lang2" to "lang1" using the update() method −

```
# Defining a set
lang1 = {"C", "C++", "Java", "Python"}
# Defining a tuple
lang2 = {"PHP", "C#", "Perl"}
lang1.update(lang2)
print (lang1)
```

The result obtained is as shown below −

```
{'Python', 'C++', 'C#', 'C', 'Java', 'PHP', 'Perl'}
```

Example

In this example, a set is constructed from a string, and another string is used as argument for update() method −

```
set1 = set("Hello")
set1.update("World")
print (set1)
```

It will produce the following output −

```
{'H', 'r', 'o', 'd', 'W', 'l', 'e'}
```

Add Set Items Using Union Operator

In Python, the union operator (|) is used to perform a union operation between two sets. The union of two sets contains all the distinct elements present in either of the sets, without any duplicates.

We can add set items using the union operator by performing the union operation between two sets using the "|" operator or union() function, which combines the elements from both sets into a new set, containing all unique elements present in either of the original sets.

Example

The following example combine sets using the union() method and the | operator to create new sets containing unique elements from the original sets −

```
# Defining three sets
lang1 = {"C", "C++", "Java", "Python"}
lang2 = {"PHP", "C#", "Perl"}
lang3 = {"SQL", "C#"}

# Performing union operation
combined_set1 = lang1.union(lang2)
```

```
combined_set2 = lang2 | lang3

# Print the combined set
print ("Combined Set1:", combined_set1)
print("Combined Set2:", combined_set2)
```

Output of the above code is as shown below −

```
Combined Set1: {'C#', 'Perl', 'C++', 'Java', 'PHP', 'Python', 'C'}
Combined Set2: {'C#', 'Perl', 'PHP', 'SQL'}
```

Example

If a sequence object is given as argument to union() method, Python automatically converts it to a set first and then performs union −

```
lang1 = {"C", "C++", "Java", "Python"}
lang2 = ["PHP", "C#", "Perl"]
lang3 = lang1.union(lang2)
print (lang3)
```

The output produced is as follows −

```
{'PHP', 'C#', 'Python', 'C', 'Java', 'C++', 'Perl'}
```

Add Set Items Using Set Comprehension

In Python, set comprehension is a way to create sets using a single line of code. It allows you to generate a new set by applying an expression to each item in an iterable (such as a list, tuple, or range), optionally including a condition to filter elements.

We can add set items using set comprehension by iterating over an iterable, applying an expression to each element, and enclosing the comprehension expression within curly braces {} to generate a new set containing the results of the expression applied to each element.

Example

In the following example, we are defining a list of integers and then using set comprehension to generate a set containing the squares of those integers −

```
# Defining a list containing integers
numbers = [1, 2, 3, 4, 5]
# Creating a set containing squares of numbers using set comprehension
squares_set = {num ** 2 for num in numbers}
# Printing the set containing squares of numbers
print("Squares Set:", squares_set)
```

Following is the output of the above code −

```
Squares Set: {1, 4, 9, 16, 25}
```

Ethical Hacking Tutorial
Docker Tutorial
Kubernetes Tutorial
DSA Tutorial
Spring Boot Tutorial
SDLC Tutorial
Unix Tutorial

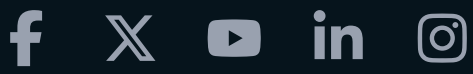
CERTIFICATIONS

Business Analytics Certification
Java & Spring Boot Advanced Certification
Data Science Advanced Certification
Cloud Computing And DevOps
Advanced Certification In Business Analytics
Artificial Intelligence And Machine Learning
DevOps Certification
Game Development Certification
Front-End Developer Certification
AWS Certification Training
Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler
Online Python Compiler
Online Go Compiler

Online C# Compiler
Online PHP Compiler
Online MATLAB Compiler
Online Bash Terminal
Online SQL Compiler
Online Html Editor



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.